Categories, tables, and grouping

Social and Cultural Analytics (7AAVBCS5)
Dr Mark J. Hill · mark.j.hill@kcl.ac.uk



KING'S College LONDON

Lecture plan

- **Part 0:** Classification as an analytical problem
- **Part I:** Categorical variables and the consequences of banding
- **Part II:** What frequency tables and crosstabs count
- **Part III:** Grouped summaries and defensible claims
- **Part IV:** Live coding and exercises

We will use London venue records and individual bike hires to learn the pandas methods and interpret their results.

Key learning outcomes

By the end of this week, you should be able to:

- distinguish nominal and ordinal categorical variables
- create frequency tables with `value_counts()`
- include and interpret missing values in tables
- create crosstabs and choose the correct percentage margin
- use `groupby()` to summarise numbers by categories
- explain how counts, rates and denominators affect cultural interpretation
- assess how classification and repeated entries limit a count of places

Part 0: Why categories matter

1. **Start with the question:** what counts as a cultural venue?
2. **Examine the labels:** how were the categories defined, and for what purpose?
3. **Check the records:** can one venue appear under several labels?
4. **Define the claim:** what does this file include, and what can its counts tell us?

From individual records to patterns

Research question: How is London's recorded cultural infrastructure distributed across venue types and boroughs?

The file has **4,434 records**, organised by 26 venue types and 33 borough areas.

However, one thing we need to check with any data is whether any of it is duplicated.

```
import pandas as pd
venues = pd.read_csv("../../data/london_cultural_venues_2023.csv")

print("Repeated row IDs:", venues["venue_id"].duplicated().sum())
print("Repeated name/borough pairs:", venues.duplicated(
    subset=["name", "borough_name"]
).sum())
```

```
Repeated row IDs: 0
Repeated name/borough pairs: 884
```

One place can appear in several category records. We will check that overlap before treating a count as a count of distinct venues.

One venue, several possible labels

The **100 Club**, located at 100 Oxford Street. It appears under three labels in our London file:

- `Music nightclubs`
- `Music venues all`
- `Music venues grassroots`

These labels describe different aspects of the same place. Counting the three entries as three separate venues would turn a classification decision into a geographical finding.



Classification as infrastructure

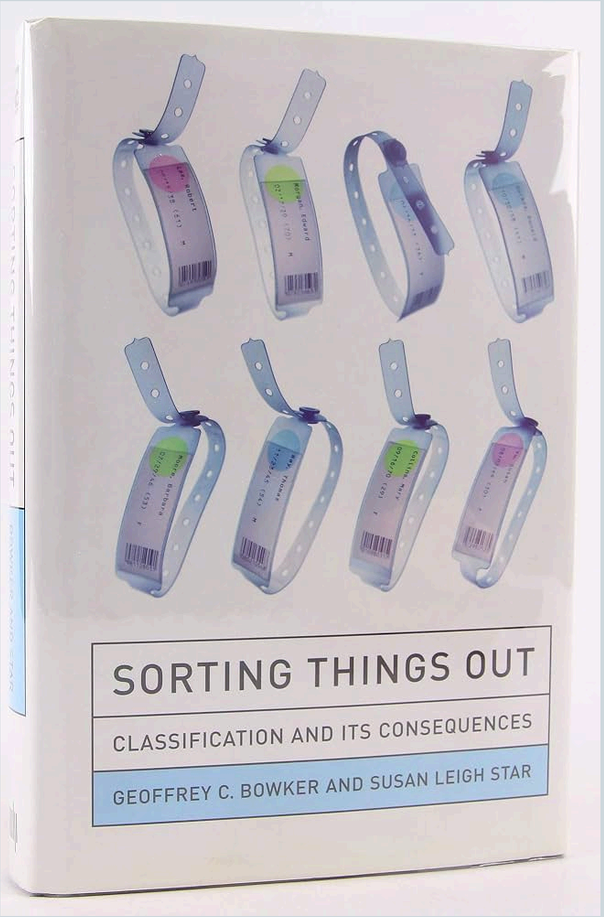
In *Sorting Things Out* (1999), Bowker & Star explain how classifications organise routines in our lives: forms require a label, databases store it, and later reports add up the labelled records.

These sorting practices are artificial, but they end up structuring our lives.

The most imfamous example for their book is “The Case of Race Classification and Reclassification under Apartheid” - reporting rules that could radically impact the lives of citizens based on *ad hoc* (and made up) racial categories.

In the London file, `venue_type` determines which entries enter a music count. `borough_name` determines which entries enter Camden’s total. A later analyst inherits those decisions even if they did not choose them.

Classification becomes infrastructure when these decisions become the ordinary machinery of counting and comparison.



Classification in practice: the 26 venue labels

Bowker & Star give three properties as an **abstract definition** of classification. They immediately explain that none of the working systems they studied satisfies all three.

Abstract property	Evidence in the London labels	What this helps us investigate
Consistent principles	Function, community, craft and production stage coexist in one list.	Which purposes do these different ways of sorting serve?
Mutual exclusivity	The 100 Club appears in three music categories.	How does multiple membership affect a count of places?
Completeness within a domain	The file has 26 selected types, with no dedicated park or community-hall category.	What does its policy scope include, and what claims fall outside it?

Departures from the abstract model are normal. The analytical interest lies in how people make categories work, handle awkward cases and live with their consequences.

The overlap is visible in the records

To check whether the 100 Club records refer to the same property, we use `uprn`: the **Unique Property Reference Number**. It is an official identifier for an addressable location, rather than a dataset row or the business using that location.

```
club = venues.loc[venues["name"] == "100 Club"]
club[["venue_type", "uprn", "venue_id"]]
```

	venue_type	uprn	venue_id
4256	Music nightclubs	10033540054	cim2023_3891
4294	Music venues all	10033540054	cim2023_3986
4346	Music venues grassroots	10033540054	cim2023_4329

The same named place and property identifier occur in three categories. The categories overlap at the level of places, even though each row has only one label.

Counting records and counting places

The 100 Club example gives us **three records of one named venue**. The three labels describe different aspects of that venue.

`Music venues all` is the name of the dataset's broad music-venue category. `Music venues grassroots` is a separate category using grassroots criteria. "All" is part of the label.

For the tables that follow, **a count means a count of rows in the file**.

Completeness within a defined scope

This does not make the dataset bad!

A dataset is (and can be) complete only in relation to **what it sets out to include**.

For example, the GLA's `Libraries` definition includes public and community libraries, but excludes libraries attached to higher education institutions.



GLA Libraries definition and data-source credit

Example	How to interpret it
A university library has no record in this category	It is outside the stated definition.
A public library meets the definition but has no record	There is a gap within the intended coverage.

Both affect what we can say about London, and may be important to our RQs. However, only the second is an omission within that definition.

How “grassroots music venue” became a category



Link to the 2015 report

A **grassroots music venue** is defined partly by its cultural role: music is central to its programme, it supports artists, and it has a place in a local music community.

The Mayor’s Music Venues Taskforce, chaired by Mark Davyd of Music Venue Trust, set out the definition in its **October 2015 Rescue Plan** (pp. 34–36).

A shared definition makes it possible to identify a group of venues, document threats to them and argue for support. The category therefore serves a practical purpose beyond describing a building.

The Rescue Plan also argued for protection. **Agent of Change** is a concrete example: under the **2021 London Plan, Policy D13**, new flats beside an existing venue must address its noise through measures such as sound insulation.

Every count is a count of a category

For each count result, keep three questions in view:

1. **What did we count?** Here, venue records. The 100 Club contributes three records, even though it is one venue.
2. **How were the categories defined?** For example, which spaces qualify as libraries, or which locations fall within a borough's boundaries?
3. **What did we divide by?** All records, records in one borough, records of one type, or residents.

The next sections show how to make those counts and comparisons in pandas.

Part I: Categorical data

1. **Distinguish the types:** categories with no inherent order, and categories with an order.
2. **Inspect the labels:** store categorical data in pandas and examine its possible values.
3. **Create and compare ordered bands:** classify borough rates as Low, Medium or High.
4. **Switch to individual hires:** create a weekday/weekend category for a new question.

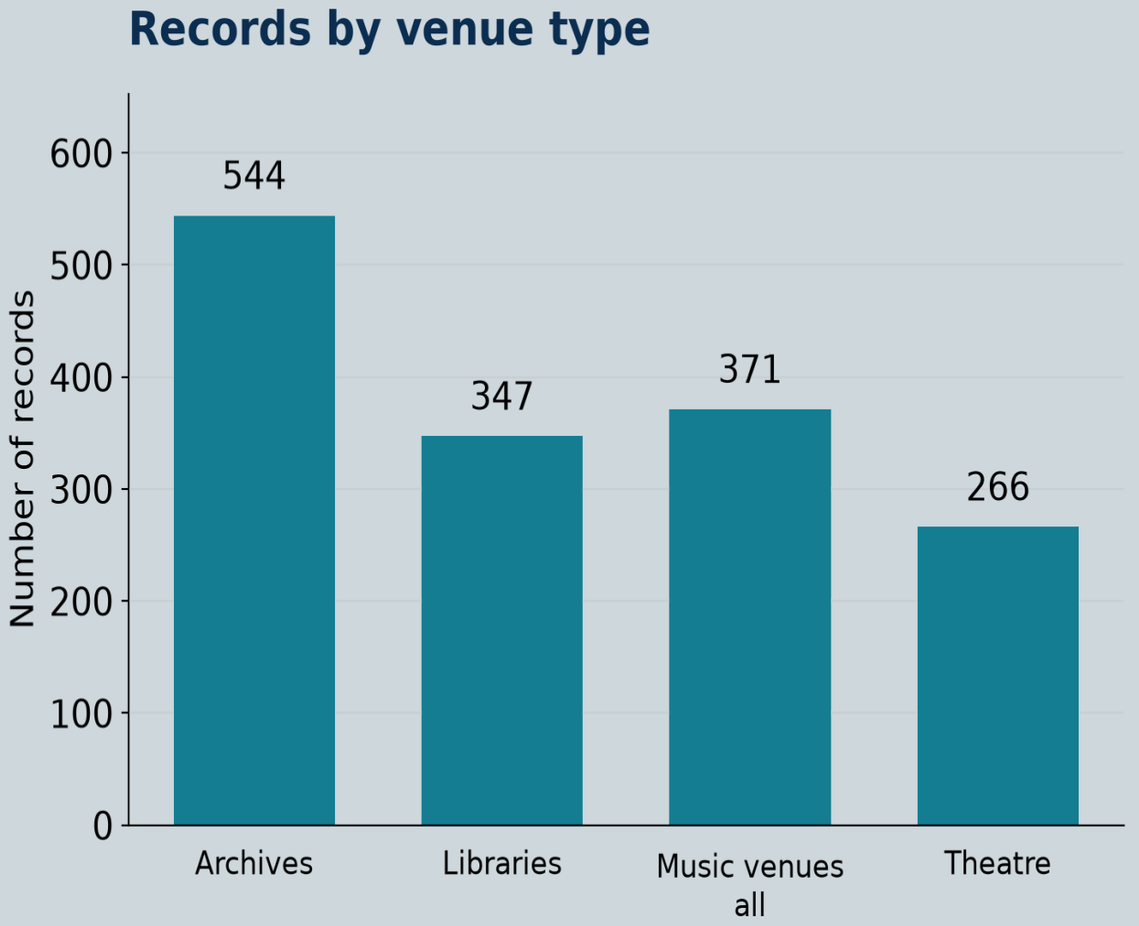
Categorical data: Nominal

Categorical variables represent **categories or groups**.

Nominal variables represent qualitative differences, with **no inherent order**. For example:

- Music genre: rock, hip-hop, classical.
- Phone operating system: iOS, Android.
- Venue type: Libraries, Theatre, Archives.

The categories are names, not numerical measurements. We can count them, but cannot calculate a meaningful average of their labels.



Four selected types from our London file. **Bar heights count records; the labels have no ranking.**

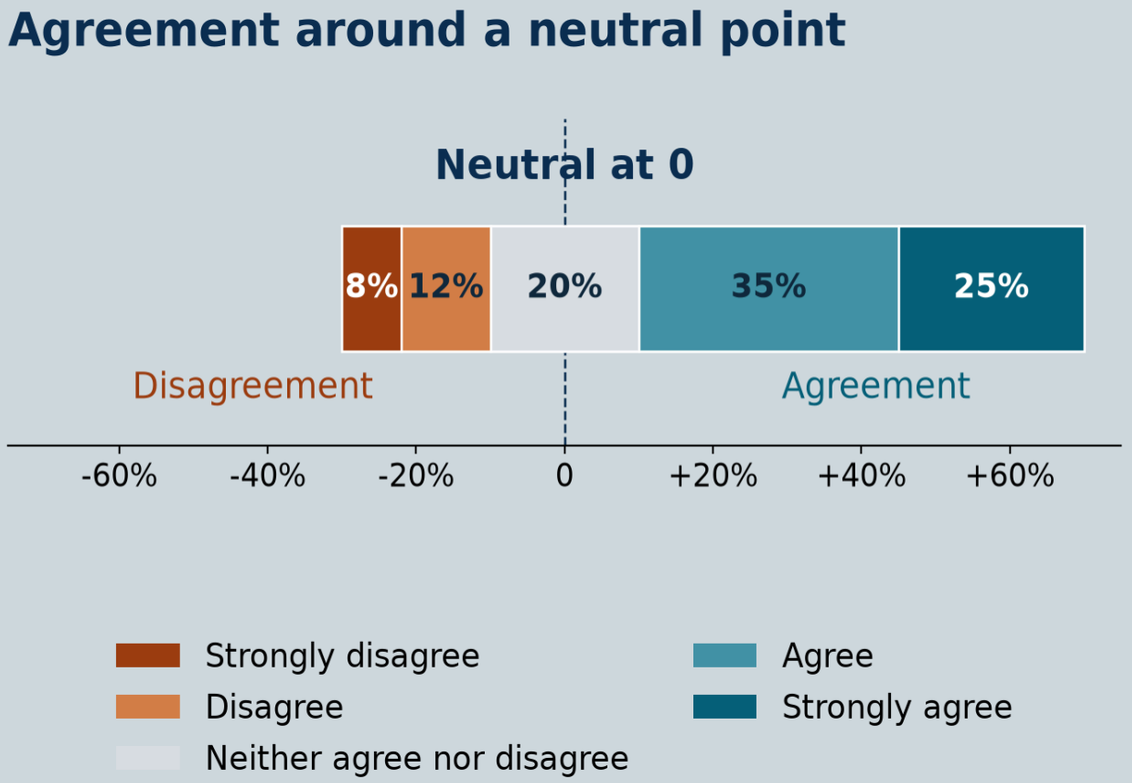
Categorical data: Ordinal

Categorical variables represent **categories or groups**.

Ordinal variables have a meaningful **order, ranking or sequence**. For example:

- Highest qualification: high school, bachelor's, master's, PhD.
- Agreement (right): "I can easily reach cultural venues from where I live." with responses coded strongly disagree, disagree, neither agree nor disagree, agree, strongly agree.

We can code these categories with numbers, such as 1–5. The order is meaningful; **equal numerical gaps are not guaranteed**.



100 fictional responses. Neutral is centred on zero; disagreement sits to the left and agreement to the right.

Python’s categorical type

Often when working with categorical data we convert it into a specific “category” datataype (dtype). In our data, that would mean turning the venue-type column to pandas’ `category` dtype.

```
1  venues["venue_type"] = venues["venue_type"].astype("category")
2  print(venues["venue_type"].cat.categories)
```

```
Index(['Archives', 'Artists workspaces', 'Arts centres', 'Cinemas',
       'Commercial galleries', 'Creative coworking desk space',
       'Creative workspaces', 'Dance performance venues',
       'Dance rehearsal studios', 'Fashion and design', 'Jewellery design',
       'LGBTQ venues', 'Libraries', 'Makerspaces', 'Making and manufacturing',
       'Museums and public galleries', 'Music nightclubs',
       'Music recording studios', 'Music rehearsal studios',
       'Music venues all', 'Music venues grassroots',
       'Prop and costume making', 'Set and exhibition building',
       'Textile design', 'Theatre', 'Theatre rehearsal studios'],
      dtype='str')
```

1. Saves memory
2. Can make operations faster
3. Makes the meaning explicit, including order
4. Preserves categories with zero observations: **This means tables and plots can show categories with a count of zero rather than omitting them entirely.**

`.astype("category")` changes how the column is stored. `.cat.categories` shows the category list.

Essential operations in pandas

Use `.cat` to work with the categories of a pandas categorical column:

Attribute / method	What it does
`venues["venue_type"].cat.categories`	Shows all possible categories.
`venues["venue_type"].cat.codes`	Shows the internal numeric code for each recorded value.
`venues["venue_type"].cat.rename_categories()`	Changes the names of existing categories. E.g., `venues["venue_type"].cat.rename_categories({"Libraries": "Public and community libraries"})`
`venues["venue_type"].cat.add_categories()`	Adds possible categories, even if no records use them yet. E.g., `venues["venue_type"].cat.add_categories(["Other"])`
`venues["venue_type"].cat.remove_unused_categories()`	Removes categories that no records currently use.

For example, `venues["venue_type"].cat.categories` lists the possible labels. `venues["venue_type"].nunique()` counts observed labels; `venues["venue_type"].value_counts()` counts records with each label.

Notes 1 and 2: codes are not values and order is not assumed

```
likert = pd.Series(["Disagree", "Neutral", "Agree"]).astype("category")
print("internal codes:", list(likert.cat.codes))
print("string values:", list(likert.astype("string")))
```

```
internal codes: [1, 2, 0]
string values: ['Disagree', 'Neutral', 'Agree']
```

The labels contain strings representing an oder. `.cat.codes` returns their positions in the category list: 1, 2 and 0.

```
numbers = pd.Series(["10", "20", "30"]).astype("category")
print("internal codes:", list(numbers.cat.codes))
print("numeric values:", list(numbers.astype("string").astype(int)))
```

```
internal codes: [0, 1, 2]
numeric values: [10, 20, 30]
```

The labels contain 10, 20 and 30. `.cat.codes` returns their positions in the category list: 0, 1 and 2.

Note 3: unused categories remain

```
havering = venues.loc[venues["borough_name"] == "Havering"]
print("possible labels:", len(havering["venue_type"].cat.categories))
print("observed types:", havering["venue_type"].nunique())
havering["venue_type"] = havering["venue_type"].cat.remove_unused_categories()
print("labels after removal:", len(havering["venue_type"].cat.categories))
```

```
possible labels: 26
observed types: 9
labels after removal: 9
```

Filtering keeps **26 possible labels**, although only **9 are recorded** in Havering's rows. Removing unused labels changes the category list, not the rows.

Making ordinal categories from borough rates

We can turn a numerical rate into an **ordered category**, just as our agreement responses had an order.

For our London data, we will group boroughs by their recorded venue rates (per 10k residents) into three bands:

Band	Records per 10,000 residents
Low	below 5
Medium	5 to below 15
High	15 or more

These cut points are our teaching choice. They are not official standards of cultural provision. This is another example of the way categories are human constructions!

The borough data for our bands

Load the borough context file. **One row now describes one borough area**, rather than one venue record.

```
borough_context = pd.read_csv(
    "../../data/london_cultural_infrastructure_2023_borough_context.csv"
)
borough_context[["borough_name", "venues_per_10k_residents"]].head()
```

	borough_name	venues_per_10k_residents
0	Westminster	28.32
1	Camden	20.36
2	Hackney	15.34
3	Southwark	9.95
4	Tower Hamlets	10.38

`venues_per_10k_residents` is a prepared rate: recorded rows per 10,000 residents. It uses 2015 population with the 2023 venue release.

Ordered categories (i.e., ordinal variables)

First assign labels using conditions, as in Weeks 3–4. Then tell pandas their order.

```
# Create a new column for our ordinal variable
borough_context["rate_band"] = ""

# Use the existing date (venues_per_10k_residents) to assign the rating to the new column
borough_context.loc[borough_context["venues_per_10k_residents"] < 5, "rate_band"] = "Low"
borough_context.loc[(borough_context["venues_per_10k_residents"] >= 5)
    & (borough_context["venues_per_10k_residents"] < 15), "rate_band"] = "Medium"
borough_context.loc[borough_context["venues_per_10k_residents"] >= 15, "rate_band"] = "High"

# Turn the column into a categorical dtype and make them ordered
borough_context["rate_band"] = borough_context["rate_band"].astype("category")
borough_context["rate_band"] = borough_context["rate_band"].cat.set_categories(
    ["Low", "Medium", "High"], ordered=True
)
print(borough_context["rate_band"].cat.categories)
print("ordered:", borough_context["rate_band"].cat.ordered)
```

```
Index(['Low', 'Medium', 'High'], dtype='str')
ordered: True
```

`.cat.set_categories(..., ordered=True)` sets **Low < Medium < High**.

The same rates, different bands

`pd.qcut()` (quantile cut) sorts the boroughs by rate and divides them into approximately equal-sized groups. Here, `q=3` requests three groups.

```
borough_context["quantile_band"], quantile_edges = pd.qcut(
    borough_context["venues_per_10k_residents"],
    q=3, labels=["Low", "Medium", "High"], retbins=True,
)
band_counts = pd.DataFrame({
    "Fixed thresholds": borough_context["rate_band"].value_counts(sort=False),
    "Quantile bands": borough_context["quantile_band"].value_counts(sort=False),
})
print(band_counts)
```

```
        Fixed thresholds  Quantile bands
Low                   22              11
Medium                 7              11
High                   4              11
```

“Low” describes a rule as well as a borough

Where do the labels change? Cultural venue entries per 10,000 residents:

	Band	Fixed rule	Quantile rule
0	Low	Below 5	Up to 1.6333
1	Medium	5 to below 15	Above 1.6333 to 4.9833
2	High	15 or above	Above 4.9833

	borough_name	venues_per_10k_residents	rate_band	quantile_band
7	Lewisham	5.05	Medium	High
23	Hillingdon	1.69	Low	Medium

Lewisham’s **5.05** is borderline low/medium with the fixed threshold but high with the quantile threshold.

London’s hire bikes: Santander Cycles

Santander Cycles is London’s docked bike-hire scheme. Hire a bike at a docking station and return it to a docking station after use.

Our data contains individual **bike hires**.

One trip record = one recorded hire = one row, with a start station, an end station, and a duration. That is, this is **non-aggregated** data.



Our bike-hire extract: one record per hire

443,539 hires starting on 16–31 July 2025, across **12 borough areas**. This is a 16-day extract, not the whole year.

```
1 trips = pd.read_csv("../../data/tfl-bike-hire/bike_trips_16-31Jul2025_with_borough.csv")
2 trips.head()
```

	trip_id	start_time	date	weekday	start_station	start_borough	end_station	end_borough	bike_model	duration_min
0	150869175	2025-07-31 23:59:00	2025-07-31	Thursday	Lambeth North Station, Waterloo	Lambeth	Kennington Road Post Office, Oval	Lambeth	CLASSIC	5.5
1	150869176	2025-07-31 23:59:00	2025-07-31	Thursday	Falkirk Street, Hoxton	Hackney	Hoxton Street, Hoxton	Hackney	CLASSIC	12.4
2	150869177	2025-07-31 23:59:00	2025-07-31	Thursday	Breams Buildings, Holborn	City of London	Hardwick Street, Clerkenwell	Islington	CLASSIC	8.6
3	150869168	2025-07-31 23:58:00	2025-07-31	Thursday	Warren Street Station, Euston	Camden	Killick Street, King's Cross	Islington	CLASSIC	17.3
4	150869169	2025-07-31 23:58:00	2025-07-31	Thursday	British Museum, Bloomsbury	Camden	Haggerston Road, Haggerston	Hackney	CLASSIC	18.3

We are going to create a **weekday/weekend category** cetegorical variableusing the existing `weekday` column.

Create a temporal category: weekday or weekend?

Question: Are recorded hires longer on weekdays or at weekends?

To compare durations, first create the groups using the existing `weekday` labels:

```
# Create a new column and populate it with "Weekday" for every row
trips["day_type"] = "Weekday"

# Use conditionals to replace "Weekday" with "Weekend" for Saturdays and Sundays.
trips.loc[
    (trips["weekday"] == "Saturday") | (trips["weekday"] == "Sunday"),
    "day_type",
] = "Weekend"

# Turn the column into a "category" datatype
trips["day_type"] = trips["day_type"].astype("category")

# Check the data
print(trips["day_type"].cat.categories)
```

```
Index(['Weekday', 'Weekend'], dtype='str')
```

`|` means **or** between conditions; `.loc[condition, "column"] = value` assigns the label to matching hires.

Weekday/weekend is nominal. The labels distinguish groups; neither is a higher rank.

Part II: Tables

1. **Count one category:** make a frequency table and identify the most common label.
2. **Calculate percentages:** express each count as a share of a stated total.
3. **Combine two categories:** count venue types within boroughs using a crosstab.
4. **Choose the comparison:** add totals, then use row or column percentages to answer different questions.

Frequency tables and the mode

A **frequency table** counts how many rows have each label. The **mode** is the most frequent label.

```
1  # Saving a very simple table as type_counts created with .value_counts()
2  type_counts = venues["venue_type"].value_counts()
3  print(type_counts.head(6))
```

```
venue_type
Archives                544
Music venues all        371
Libraries               347
Commercial galleries    300
Artists workspaces      270
Theatre                 266
Name: count, dtype: int64
```

```
1  # "Averages" with categorical data: the mode
2  print("mode:", type_counts.idxmax())
```

```
mode: Archives
```

`value_counts()` sorts the largest count first. `.head(6)` displays six rows; `.idxmax()` returns the label attached to the largest count.

Counts and percentages

A percentage expresses a category's count as a share of the included records.

```
type_percent = venues["venue_type"].value_counts(normalize=True) * 100
print(type_percent.round(1).head(6))
```

```
venue_type
Archives                12.3
Music venues all         8.4
Libraries                7.8
Commercial galleries     6.8
Artists workspaces       6.1
Theatre                  6.0
Name: proportion, dtype: float64
```

For Archives: **544 ÷ 4,434 × 100 = 12.3%**. The denominator is all records with a venue-type label, not just the six types displayed.

Use the time category: how many hires?

Count the labels we created, then express them as shares of the recorded hires.

```
1  print(trips["day_type"].value_counts())
2  print((trips["day_type"].value_counts(normalize=True) * 100).round(1))
```

```
day_type
Weekday    358643
Weekend     84896
Name: count, dtype: int64
day_type
Weekday    80.9
Weekend    19.1
Name: proportion, dtype: float64
```

358,643 weekday hires (80.9%) and **84,896 weekend hires (19.1%)**.

But the extract contains **12 weekdays and four weekend days**. These totals cover different numbers of days.

To compare typical daily activity, account for the days represented; to compare hire duration, summarise the individual durations in Part III.

A readable borough by type crosstab

A **crosstab** counts combinations of two labels. Here rows are borough areas and columns are venue types.

```
# Create a crosstab of boroughs and venue types
borough_type_counts = pd.crosstab(
    venues["borough_name"], venues["venue_type"],
)

# Rather than displaying the entire crosstab (which would be 33x26) we choose 3 boroughs and 3 venue types to s
shown_boroughs = ["Camden", "Hackney", "Westminster", "Havering"]
shown_types = ["Libraries", "Music venues all", "Music nightclubs"]
borough_type_counts.loc[shown_boroughs, shown_types]
```

venue_type	Libraries	Music venues all	Music nightclubs
borough_name			
Camden	12	36	9
Hackney	9	71	31
Westminster	11	52	25
Havering	10	0	0

Crosstab with margins

Margins are totals: a total for each row, a total for each column and a grand total.

```
with_totals = pd.crosstab(
    venues["borough_name"], venues["venue_type"],
    margins=True, margins_name="Total",
)
with_totals.loc[shown_boroughs + ["Total"], shown_types + ["Total"]]
```

venue_type	Libraries	Music venues all	Music nightclubs	Total
borough_name				
Camden	12	36	9	469
Hackney	9	71	31	403
Westminster	11	52	25	653
Havering	10	0	0	22
Total	347	371	133	4434

The totals include **all types and all London borough areas**, including those hidden from this display. Camden's total is 469; the grand total is 4,434.

With percentages - but which margin?

The Camden music count is **36**. If we want that as a percentage is is percentage of venues in Camden or percentage of Camden venues that make up London. The denominator determines which question the percentage answers.

Question about `Music venues all` records	Calculation	Option
What share of Camden's records have this label?	36 ÷ 469 (total venues in Camden) = 7.7%	`normalize="index"`
What share of London's records with this label are in Camden?	36 ÷ 371 (total music venues in London) = 9.7%	`normalize="columns"`

`index` means the table's rows. `columns` means the venue-type columns.

Row percentages: Camden's venue mix

Question: What share of Camden's records are labelled `Music venues all`?

```
row_percent = pd.crosstab(
    venues["borough_name"], venues["venue_type"],
    normalize="index",
) * 100
row_percent.loc[shown_boroughs, shown_types].round(1)
```

venue_type	Libraries	Music venues all	Music nightclubs
borough_name			
Camden	2.6	7.7	1.9
Hackney	2.2	17.6	7.7
Westminster	1.7	8.0	3.8
Havering	45.5	0.0	0.0

Camden: **36 of 469 records = 7.7%**. Each complete borough row sums to 100% across all 26 types; the three displayed columns are only part of that row.

Column percentages: music venues across London

Question: What share of London's `Music venues all` records are in Camden?

```
column_percent = pd.crosstab(
    venues["borough_name"], venues["venue_type"],
    normalize="columns",
) * 100
column_percent.loc[shown_boroughs, shown_types].round(1)
```

venue_type	Libraries	Music venues all	Music nightclubs
borough_name			
Camden	3.5	9.7	6.8
Hackney	2.6	19.1	23.3
Westminster	3.2	14.0	18.8
Havering	2.9	0.0	0.0

Camden: **36 of 371 records = 9.7%**. Each complete venue-type column sums to 100% across London; the four displayed boroughs are only part of that column.

Part III: Grouping

1. **Identify the grouping variable:** which categories am I comparing?
2. **Select the numerical variable:** what am I summarising?
3. **Choose a calculation:** count, sum, mean or median.
4. **Read the result:** one summary per group, and explain what each number means.

Our grouping question: where do hires start, and how long do they last?

Return to `trips`: **one record is one hire**, with a starting borough and a recorded duration in minutes.

```
1  trips[["start_borough", "end_borough", "day_type", "duration_min"]].head(3)
```

	start_borough	end_borough	day_type	duration_min
0	Lambeth	Lambeth	Weekday	5.5
1	Hackney	Hackney	Weekday	12.4
2	City of London	Islington	Weekday	8.6

Decision	Our choice
Grouping variable	`start_borough`
Numerical variable	`duration_min`
Calculations	Count hires; compare mean and median duration

The result is one summary of `duration_min` per `start_borough`.

Step 1: group hires by starting borough

Put hires with the same `start_borough` together, then count them.

```
# Create the "groupby" obeject
borough_groups = trips.groupby("start_borough")

# Print the size of each category
print(borough_groups.size())
```

```
start_borough
Camden                    39135
City of London            28988
Hackney                   20434
Hammersmith and Fulham    24390
Islington                 18256
Kensington and Chelsea    44598
Lambeth                   32875
Newham                     4400
Southwark                 32810
Tower Hamlets             60677
Wandsworth                25706
Westminster              111270
dtype: int64
```

`groupby("start_borough")` defines the groups. `.size()` counts all records in each group.

Steps 2 and 3: select duration and calculate its median

Question: What is the median recorded hire duration for each starting borough?

```
1  # Now we can use .median() to calculate the median duration time.
2  median_duration = borough_groups["duration_min"].median()
3  print(median_duration)
```

```
start_borough
Camden                    13.4
City of London            13.1
Hackney                   12.3
Hammersmith and Fulham    12.7
Islington                 11.8
Kensington and Chelsea    14.3
Lambeth                   13.0
Newham                    24.0
Southwark                 13.4
Tower Hamlets             12.3
Wandsworth                14.1
Westminster               16.6
Name: duration_min, dtype: float64
```

`["duration_min"]` selects the column to summarise. `.median()` finds its *middle* value **within each group**.

For hires starting in Westminster, the median is **16.6 minutes**; for Tower Hamlets, **12.3 minutes**, for Newham is **24 minutes**

EDA normally reveals awkward values

Exploratory data analysis (EDA) includes checking the data beind the measurements we want to summarise.

```
1  # Use .describe to get an overview of the data distribution for trip durations
2  print(trips["duration_min"].describe().round(1))
```

```
count    443539.0
mean         23.0
std         231.3
min           0.0
25%           8.1
50%          14.0
75%          23.0
max       75882.8
Name: duration_min, dtype: float64
```

Anything odd here?

EDA normally reveals awkward values II

Exploratory data analysis (EDA) includes checking the data beind the measurements we want to summarise.

```
1  # There are clear instances of trips of
2  # zero length and longer than 24 hours.
3  print("Zero durations:",
4      (trips["duration_min"] == 0).sum())
5  print("Over 24 hours:",
6      (trips["duration_min"] > 1440).sum())
```

```
Zero durations: 199
Over 24 hours: 269
```

There are **199 zero durations** and **269 over 24 hours**. The largest is about **53 days**!

These are values to investigate. **Check the source and document decisions before changing them.**



Boris bike on Mont Ventoux

“On October 26 2013, three friends took on Mont Ventoux on a Boris bike”

Compare count, mean, and median in one table

Apply several calculations to the **same duration values within each borough**.

```
1  duration_summary = borough_groups["duration_min"].agg(["count", "sum", "mean", "median"])
2  duration_summary.round(1)
```

start_borough	count	sum	mean	median
Camden	39135	834821.8	21.3	13.4
City of London	28988	521856.0	18.0	13.1
Hackney	20434	415807.1	20.3	12.3
Hammersmith and Fulham	24390	516841.4	21.2	12.7
Islington	18256	297992.1	16.3	11.8
Kensington and Chelsea	44598	1116005.5	25.0	14.3
Lambeth	32875	664102.0	20.2	13.0
Newham	4400	183944.9	41.8	24.0
Southwark	32810	702487.5	21.4	13.4
Tower Hamlets	60677	1308398.3	21.6	12.3
Wandsworth	25706	613351.9	23.9	14.1
Westminster	111270	3011087.5	27.1	16.6

`.agg(["count", "mean", "median"])` applies each named calculation. `count` counts non-missing durations; `mean` and `median` are in minutes.

Reuse the time category: compare hire durations

Question: Do recorded hire durations differ between weekday and weekend starts?

Change the **grouping column**; keep the numerical variable and calculations the same.

```
day_type_summary = trips.groupby("day_type")["duration_min"].agg(["count", "mean", "median"])
day_type_summary.round(1)
```

day_type	count	mean	median
Weekday	358643	22.1	13.7
Weekend	84896	26.8	15.2

The median is **13.7 minutes for weekday hires** and **15.2 minutes for weekend hires** in this extract.

`groupby("day_type")` uses the category we created in Part I. These are **unfiltered recorded durations**, not a measure of speed or trip purpose.

Step 4: say what the result means

A supported finding:

> Among recorded hires starting on **16–31 July 2025**, the median duration on weekdays was **36.7 minutes**, compared with **15.2 minutes** for trips on weekends. Thus, trips are longer on weekends, depite there being fewer of them.

Notes:

- The unit of anlaysis is a **hired trip**, not a distinct rider.
- The groups describe **starting locations**, not riders' home boroughs.
- Recorded duration does not establish **distance, speed, or trip purpose**.

State the measure, the groups and the period. Use the EDA checks to explain the limits of the claim.

Group by two variables: starting and ending borough

Question: Which borough-to-borough pairs appear most often in the hire records?

```
1  borough_flows = trips.groupby(["start_borough", "end_borough"]).size()
2  print(borough_flows.sort_values(ascending=False))
```

```
start_borough            end_borough
Westminster              Westminster                55685
Tower Hamlets            Tower Hamlets              35869
Kensington and Chelsea   Kensington and Chelsea     17531
                         Westminster                13846
Westminster              Kensington and Chelsea     13563
                                                    ...
Kensington and Chelsea   Newham                         7
Newham                   Wandsworth                     7
Wandsworth               Newham                         4
Hammersmith and Fulham   Newham                         3
Newham                   Hammersmith and Fulham         1
Length: 144, dtype: int64
```

`groupby(["start_borough", "end_borough"])` uses **both categories**. Each result counts hires with that combination of endpoints.

Useful operations on a GroupBy object

We already have `borough_groups = trips.groupby("start_borough")`. Choose an operation to answer your question.

Counts and categories

Method / attribute	What it gives
`.size()`	Number of records, including those with missing measurements
`.count()`	Number of non-missing values in each selected column
`.nunique()`	Number of distinct non-missing values
`.value_counts()`	Frequency of each label in a selected column
`.ngroups`	Number of groups in the GroupBy object; no parentheses

Numerical summaries

Method	What it gives within each group
`.sum()`	Total of the selected values
`.mean()`, `.median()`	Average or middle value
`.min()`, `.max()`	Smallest or largest value
`.describe()`	Several summary statistics
`.agg([...])`	The calculations you list, such as count, sum, mean and median

Select the column you want to summarise: `borough_groups["duration_min"].mean()` or `borough_groups["end_station"].nunique()`.

Later questions: movement and cultural places

We’ve got quite a lot of data now. Let’s spend some time thinking about what types of questions we might be able to ask and answer with it.

Where this is going

Question	Table method	A later visual form
How many records have each label?	`value_counts()`	Bar plot
How do type and borough occur together?	`pd.crosstab()`	Grouped or proportional bar plot
How does a number vary by category?	`groupby().agg()`	Comparison of group summaries
How is one number distributed?	`describe()`	Histogram

The next lecture turns tables into charts. Categories, counting units, and denominators still determine what the chart can say.

Part IV: Live coding and exercises

Exercise 1: categories

Using the venue records already loaded as `venues`:

1. Identify `venue_type` and `borough_name` as nominal variables.
2. Convert venue type to `category` and inspect its possible labels.
3. Filter to Havering. Compare possible labels with observed types.
4. Remove unused labels. Check what changed and what stayed the same.
5. Give one sentence explaining why a proposed split or merge of types would affect a count.
6. Switch to `trips`: create the nominal weekday/weekend category using its `weekday` column.

Exercise 1 continued: entries and places

How many records? How many places can we identify?

1. Check missing `uprn` values: count them and calculate their percentage.
2. Compare `len(venues)` with `venues["uprn"].nunique()`. Remember that `nunique()` excludes blanks.
3. Inspect `venues["uprn"].value_counts().head(10)` for placeholders and scientific notation.
4. Revisit the 100 Club's records, names and property number.
5. Write one sentence explaining why neither count establishes the number of operating venues.

A UPRN identifies an addressable location. Several organisations can share a property.

Exercise 2: tables

1. Count venue-type labels and identify the mode.
2. Express those counts as percentages and state what the denominator includes.
3. Make a full borough-by-type crosstab, then display a few rows and columns.
4. Add totals, remembering that hidden types and boroughs still contribute.
5. Compare Camden’s row and column music percentages. Complete one sentence for each question.
6. Use `reset_index()` to make borough an ordinary column.

Exercise 3: grouping bike hires

Use `trips`, the bike-hire data introduced in Part I.

1. Group hires by `start_borough` and count them with `.size()`.
2. Select `duration_min` and find the median within each group.
3. Inspect the durations and identify one unusual value to investigate.
4. Use `.agg(["count", "mean", "median"])` to compare the summaries.
5. Repeat the duration summary using `day_type` as the grouping column.
6. Write one comparison sentence, naming the measure, groups and period.

Extra practice: count starting/ending borough pairs. Which different-borough pair has the most hires?

Activity: work with your data

If you have time, use your project dataset:

- Choose one categorical column and inspect its definition and missing values.
- Make one frequency table, crosstab or grouped summary relevant to your question.
- State the counting unit and, if using a percentage, the denominator.
- Write one sentence about what the result shows and one thing it cannot establish.

Who created the category, and how does its definition affect this result?